

Data preparation in R

Eliot Monaco

Population health scientist
Kansas City Health Department

2025-12-03

Overview

Parts of an analysis

- **Data prep:** Get data ready for analysis
- **Analyze data:** Ask questions of the data and get answers
- **Create output(s):** Format the results for external consumption

What is data prep?

- **Validate:** Is my data what I expect it to be?
- **Clean:** What corrections need to be made for the data to be useful?
- **Configure:** What modifications need to be made for the purposes of my analysis?

Rows and columns

- Each row is a unique **observation**
- Each column is a distinct **variable**

Columns (variables)

- All values in a column are of the same **type**
- Common types:
 - numeric (or double or integer)
 - character (or string or text)
 - date (or POSIX)

Columns (variables)

animal	age	favorite_food	birth_date
dog	4	kibble	2021-04-10
cat	10	fancy feast	2015-11-18
fish	0.5	flakes	2025-05-22

Data validation

How do I know what to expect?

A **data dictionary** tells you which values are allowed.

name	description	type	allowed_values
state_fips	State FIPS code	text	2-digit number
manner	Manner of death code	text	A = Accident, S = Suicide, H = Homicide, P = Pending Investigation, N = Natural, C = Could not be determined

How do I know what to expect?

If there is no data dictionary, use context.

- Other documentation from the data source
- Column names
- Column values

Functions for snapshotting dataframes

- `head()`
- `str()`
- `glimpse()`

head()

```

1 library(dplyr)
2
3 head(starwars)

# A tibble: 6 × 14
  name      height mass hair_color skin_color eye_color birth_year sex   gender homeworld species
<chr> <dbl> <dbl> <chr> <chr> <chr> <dbl> <chr> <chr> <chr> <chr>
<lis> <list> <list> <list>
1 Luke ... 172 77 blond fair      gold      blue      19 male mascul... Tatooine Human
<chr> <chr> <chr> <chr [2]>
2 C-3PO 167 75 <NA> gold      yellow    112 none mascul... Tatooine Droid
<chr> <chr> <chr [0]>
3 R2-D2 96 32 <NA> white, bl... red      33 none mascul... Naboo Droid
<chr> <chr> <chr [0]>
4 Darth... 202 136 none white      yellow    41.9 male mascul... Tatooine Human
<chr> <chr> <chr [1]>
5 Leia ... 150 49 brown light      brown     19 fema... femin... Alderaan Human
<chr> <chr> <chr [0]>

```

str()

```

1 library(dplyr)
2
3 str(starwars)

tibble [87 × 14] (S3: tbl_df/tbl/data.frame)
 $ name      : chr [1:87] "Luke Skywalker" "C-3PO" "R2-D2" "Darth Vader" ...
 $ height    : int [1:87] 172 167 96 202 150 178 165 97 183 182 ...
 $ mass      : num [1:87] 77 75 32 136 49 120 75 32 84 77 ...
 $ hair_color: chr [1:87] "blond" NA NA "none" ...
 $ skin_color: chr [1:87] "fair" "gold" "white, blue" "white" ...
 $ eye_color : chr [1:87] "blue" "yellow" "red" "yellow" ...
 $ birth_year: num [1:87] 19 112 33 41.9 19 52 47 NA 24 57 ...
 $ sex       : chr [1:87] "male" "none" "none" "male" ...
 $ gender    : chr [1:87] "masculine" "masculine" "masculine" "masculine" ...
 $ homeworld : chr [1:87] "Tatooine" "Tatooine" "Naboo" "Tatooine" ...
 $ species   : chr [1:87] "Human" "Droid" "Droid" "Human" ...
 $ films     :List of 87
 ..$ : chr [1:5] "A New Hope" "The Empire Strikes Back" "Return of the Jedi" "Revenge of the
Sith" ...

```

glimpse()

```

1 library(dplyr)
2
3 glimpse(starwars)

Rows: 87
Columns: 14
 $ name      <chr> "Luke Skywalker", "C-3PO", "R2-D2", "Darth Vader", "Leia Organa", "Owen Lars",
"Beru Whitesun Lars"...
 $ height    <int> 172, 167, 96, 202, 150, 178, 165, 97, 183, 182, 188, 180, 228, 180, 173, 175,
170, 180, 66, 170, 18...
 $ mass      <dbl> 77.0, 75.0, 32.0, 136.0, 49.0, 120.0, 75.0, 32.0, 84.0, 77.0, 84.0, NA, 112.0,
80.0, 74.0, 1358.0, ...
 $ hair_color <chr> "blond", NA, NA, "none", "brown", "brown, grey", "brown", NA, "black",
"auburn, white", "blond", "a...
 $ skin_color <chr> "fair", "gold", "white, blue", "white", "light", "light", "light", "white,
red", "light", "fair", "...
 $ eye_color  <chr> "blue", "yellow", "red", "yellow", "brown", "blue", "blue", "red", "brown",
"blue-gray", "blue", "b...
 $ birth_year <dbl> 19.0, 112.0, 33.0, 41.9, 19.0, 52.0, 47.0, NA, 24.0, 57.0, 41.9, 64.0, 200.0,
200.0, 44.0, 200.0, 21

```

Global checks

- What are the column names?
- Is the number of rows unexpected?
- Are there duplicate rows?
- Are there missing values?

Checking individual variables

Which variables are necessary to validate for the analysis? You may not need to validate them all.

Useful things

- `dplyr::count()`
- `dplyr::filter()`
- Regular expressions

count()

Compare column values against what's expected.

```
1 starwars |>
2   count(hair_color)

# A tibble: 12 x 2
  hair_color      n
  <chr>      <int>
1 auburn          1
2 auburn, grey    1
3 auburn, white    1
4 black          13
5 blond           3
6 blonde          1
7 brown          18
8 brown, grey     1
9 grey            1
10 none          38
11 white           4
12 <NA>           5
```

filter()

Seek out unexpected values.

```
1 df <- sf::st_drop_geometry(kcData::sf_tract_2024)
2
3 # Check column for a single value
4 df |>
5 filter(STATEFP != "29") |>
6 nrow()
[1] 0
```

```
1 # Check column for multiple values
2 df |>
3 filter(!COUNTYFP %in% c("037", "047", "095", "165")) |>
4 nrow()
[1] 0
```

filter() + regex

Seek out unexpected values.

```
1 # Check column based on a regular expression pattern
2 df |>
3   filter(!grepl("^\\d{6}$", TRACTCE)) |>
4   nrow()
[1] 0
```

What have we done so far?

So far, we haven't changed the data in any way. We are just playing detective and **learning** about the data.

Data cleaning

Important principles

- Do everything in code
- Do not manually change the source data (e.g., editing in Excel)
- Do not overwrite the source data - keep it in its wild, shaggy state
- Save your modified data as a separate file
- Document and comment extensively

Example: Addresses

Cleaning addresses for geocoding. We need street addresses only.

street	city	state	zip
123 Main St.	Kansas City	MO	64102
Bob's house, 123 Main St.	Kansas City	MO	64102
123 Main St.	Kansas City	MO	

```
1 addresses <- tribble(  
2   ~street, ~city, ~state, ~zip,  
3   "123 Main St.", "Kansas City", "MO", "64102",  
4   "Bob's house, 123 Main St.", "Kansas City", "MO", "64102",  
5   "", "123 Main St.", "Kansas City", "MO"  
6 )
```

street variable

```
1 # `street` doesn't start with a digit
2 addresses |>
3 filter(!grepl("^\\d", street))
```

```
# A tibble: 2 × 4
  street                city      state      zip
<chr>                <chr>    <chr>    <chr>
1 "Bob's house, 123 Main St." Kansas City MO    64102
2 ""                  123 Main St. Kansas City MO
```

```
1 # `street` starts with a non-digit
2 addresses |>
3 filter(grepl("^\\D", street))
```

```
# A tibble: 1 × 4
  street                city      state      zip
<chr>                <chr>    <chr>    <chr>
1 Bob's house, 123 Main St. Kansas City MO    64102
```

city variable

```
1 # `city` doesn't contain "kansas city" (case insensitive)
2 addresses |>
3 filter(!grepl("(?)kansas city", city))
```

```
# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 ""      123 Main St. Kansas City MO
```

```
1 # `city` contains a digit
2 addresses |>
3 filter(grepl("\\d", city))
```

```
# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 ""      123 Main St. Kansas City MO
```

```
1 # `city` is not in a list of allowed cities
2 cities <- c("Kansas City", "St. Louis", "Chicago")
3
4 addresses |>
5 filter(!city %in% cities)
```

```
# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 " 123 Main St. Kansas City MO
```

state variable

Can use similar methods as with `city`.

- `state != "MO"`
- `!state %in% states`

```
1 # `state` is not two uppercase letters
2 addresses |>
3 filter(!grepl("^[[:upper:]]{2}$", state))
```

```
# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 "      123 Main St. Kansas City MO
```

Regex detour

```

1 states <- c("MO", "KS", "WI", "CA", "CAT", "DOG", "lEtTERS")
2
3 # Without start/end tokens
4 result <- grepl("[[:upper:]]{2}", states)
5
6 setNames(result, states)

```

MO	KS	WI	CA	CAT	DOG	lEtTERS
TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

```

1 # With start/end tokens
2 result <- grepl("^[[[:upper:]]{2}$", states)
3
4 setNames(result, states)

```

MO	KS	WI	CA	CAT	DOG	lEtTERS
TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE

zip variable

```
1 # `zip` is not 5 digits
2 addresses |>
3   filter(!grepl("^\\d{5}$", zip))

# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 ""      123 Main St. Kansas City MO
```

```
1 # `zip` is not 5 digits starting with "64"
2 addresses |>
3   filter(!grepl("^64\\d{3}$", zip))

# A tibble: 1 × 4
  street city      state      zip
<chr> <chr> <chr> <chr>
1 ""      123 Main St. Kansas City MO
```

Data import detour

When importing data, be careful that you don't change it unexpectedly!

- If a variable can have leading zeros, import as character
- Dates, times, and date-time values can be read in many different ways
- Probably other examples

Data import functions

Know how data import functions read in data and determine types.

For example, `readr::read_csv()`.

Example: ICD codes

ICD-10 codes have a specific format.

Example: “W56” (Contact with marine animal)

- Necessary part: uppercase letter + two digits
- Optional part: period + additional digits

How to check?

- Use a list of allowed values
- Use regex, e.g., `^[[:upper:]]\\d{2}(\\.\\d{1,5})?.$`

```
1 p <- "^[[:upper:]]\\d{2}(\\.\\d{1,5})?.$"  
2
```

```
3 icd <- c("C41", "C41.9", "C41.999", "C41.", "C4.5", "C41.5x")
```

```
4
```

```
5 result <- grepl(p, icd)
```

```
6
```

```
7 setNames(result, icd)
```

C41	C41.9	C41.999	C41.	C4.5	C41.5x
TRUE	TRUE	TRUE	FALSE	FALSE	FALSE

Example: ICD codes

```
1 vital <- readRDS("icd_example.rds")
2
3 vital |>
4 mutate(across(matches("cod|mult"), ~ if_else(is.na(.x), "", .x))) |>
5 head() |>
6 gt::gt()
```

rowid	cod	mult01	mult02	mult03	mult04	mult05	mult06	mult07	mult08	mult09	mult10	mult11	mult12	mult13	mult14	mult15	mult16	mult17	mult18	mult19	mult20
1	C259	C259																			
2	F03	F03																			
3	E889	E889	I272	I509	J90 J	960 R	090														
4	C509	C509	C795	I509	J960	J961	N183														
5	X95	X95 T	019 T	141																	
6	C64	C64 B	348 C	782 D	649 E	875 J	129 R	090													

Notable features

- No periods (this is okay)
- Some values are normal, but...
- Some have a space + an extra letter
- Some have just numbers

What is going on?

ICD codes are split across columns.

Fix with a custom function

```
1 # Fix problems in VR ICD code variables
2 fix_vital_icd <- function(df, row_id) {
3   # Check uniqueness of row ID variable
4   if (any(duplicated(df[[row_id]]))) {
5     stop("`row_id` is not unique")
6   }
7
8   # Filter `df` for invalid ICD codes
9   df2 <- df |>
10    dplyr::filter(dplyr::if_any(
11      c(cod, tidyselect::starts_with("mult")),
12      ~ !stringr::str_detect(.x, "^[alpha:]\d{2,4}$")
13    ))
14
15 # Variables with ICD codes
16 var <- stringr::str_which(colnames(df), "(?i)^cod$|^mult\\d{2}$")
17
18 # Roll across columns with ICD codes and fix
```

Fix with a custom function

```

1 vital2 <- fix_vital_icd(vital, "rowid")
2
3 vital2 |>
4 mutate(across(matches("cod|mult"), ~ if_else(is.na(.x), "", .x))) |>
5 head() |>
6 gt::gt()

```

rowid	cod	mult01	mult02	mult03	mult04	mult05	mult06	mult07	mult08	mult09	mult10	mult11	mult12	mult13	mult14	mult15	mult16	mult17	mult18	mult19	mult20
1	C259	C259																			
2	F03	F03																			
3	E889	E889	I272	I509	J90	J960	R090														
4	C509	C509	C795	I509	J960	J961	N183														
5	X95	X95	T019	T141																	
6	C64	C64	B348	C782	D649	E875	J129	R090													

Wrap-up

- Data prep is about getting to know your data, knowing what you should expect from your data, and questioning your assumptions about the data
- Data prep involves a lot of exploration and testing different ways of doing things
- Know how your code is handling values you might not be considering: NA, empty strings, negative numbers
- tidyverse is very useful (I recommend also knowing how to accomplish the same step in base R)
- Regular expressions are extremely helpful

Resources

- [stringr](#) website and cheatsheet
- [regex](#) cheatsheet
- [Regular-Expressions.info](#)